Questionnaire Input Type:

Feature Name	Type	Possible Values (COMPLETE)
gender	string	Male, Female
age	integer	18–70
Investment_Avenues	string	Yes, No
Mutual_Funds	integer	1–7
Equity_Market	integer	1–7
Debentures	integer	1–7
Government_Bonds	integer	1–7
Fixed_Deposits	integer	1–7
PPF	integer	1–7
Gold	integer	1–7
Stock_Marktet	string	Yes, No
Factor	string	Risk, Returns, Locking Period
Objective	string	Risk, Returns, Growth, Income
Purpose	string	Wealth Creation, Savings for Future, Returns, Income
Duration	string	Less than 1 year, 1-3 years, 3-5 years, More than 5 years
Invest_Monitor	string	Monthly, Weekly, Daily
Expect	string	10%-20%, 20%-30%, 30%-40%
Avenue	string	Public Provident Fund, Mutual Fund, Equity, Fixed Deposits

What are your savings objectives?	string	Health Care, Retirement Plan, Education
Reason_Equity	string	Capital Appreciation, Dividend, Liquidity
Reason_Mutual	string	Better Returns, Fund Diversification, Tax Benefits
Reason_Bonds	string	Assured Returns, Safe Investment, Tax Incentives
Reason_FD	string	Fixed Returns, Risk Free, High Interest Rates
Source	string	Newspapers and Magazines, Financial Consultants, Television, Internet

Model Artifact File Format and Loader Code

File: risk_tolerance_model.pkl

Format: Pickle (joblib) containing a dict with model + preprocessing objects.

Loader Code (load_model.py)

```
import joblib
import pandas as pd

def load_risk_model(path="risk_tolerance_model.pkl"):
    artifact = joblib.load(path)
    return artifact

def predict_risk(input_dict: dict, artifact):
    df = pd.DataFrame([input_dict])

    # Rank reversal
    rank_cols = ['Mutual_Funds', 'Equity_Market', 'Debentures', 'Government_Bonds',
                 'Fixed_Deposits', 'PPF', 'Gold']
    for col in rank_cols:
        if col in df.columns:
```

```
            df[col] = 8 - pd.to_numeric(df[col], errors='coerce')

    # Score mappings
    duration_map = {'Less than 1 year':1, '1-3 years':2, '3-5 years':3, 'More than 5 years':4}
    expect_map = {'10%-20%':1, '20%-30%':2, '30%-40%':3}
    monitor_map = {'Monthly':1, 'Weekly':2, 'Daily':3}

    df['Duration_Score'] = df['Duration'].map(duration_map).fillna(0)
    df['Expect_Score'] = df['Expect'].map(expect_map).fillna(0)
    df['Monitor_Score'] = df['Invest_Monitor'].map(monitor_map).fillna(0)

    # One-hot encoding (all nominal columns)
    nominal_cols = ['gender','Objective','Purpose','What are your savings
objectives?','Stock_Marktet']
    df = pd.get_dummies(df, columns=[c for c in nominal_cols if c in df.columns], drop_first=True)

    # Align with training features
    feature_names = artifact['feature_names']
    for col in feature_names:
        if col not in df.columns:
            df[col] = 0
    df = df[feature_names]

    X_scaled = artifact['scaler'].transform(df)
    model = artifact['model']
    pred = int(model.predict(X_scaled)[0])
    proba = model.predict_proba(X_scaled)[0]

    label_names = artifact.get('label_names', {0:'Conservative', 1:'Moderate', 2:'Aggressive'})
    risk_mapping = {0: "Low", 1: "Medium", 2: "High"}

    return {
        "risk_class": pred,
        "risk_label": label_names[pred],
        "risk_level": risk_mapping[pred],
        "probabilities": {label_names[i]: float(p) for i, p in enumerate(proba)},
        "risk_score": float(proba[2] * 100)
    }
—----------------------------------------------------
```

Python and Package Versions Required for Inference

File: requirements.txt

```
pandas==2.2.2
```

numpy==1.26.4
scikit-learn==1.5.1
xgboost==2.1.1
catboost==1.2.7
joblib==1.4.2

—------

Mapping from Model Output to Low, Medium, and High Risk Levels

- 0 → **Low** (Conservative)
- 1 → **Medium** (Moderate)
- 2 → **High** (Aggressive)